Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ ПОДДЕРЖКИ YDB В СИСТЕМЕ KEYCLOAK В КАЧЕСТВЕ ОСНОВНОГО ХРАНИЛИЩА ДАННЫХ

Автор: Мухтаров Айнур Дамирович _____

Направление подготовки: 01.03.02 Прикладная математика и информатика

Квалификация: Бакалавр

Руководитель ВКР: Аксенов В.Е., д.т.н. _____

Санкт-Петербург, 2026 г.

Обучающийся Мухтаров Айнур Дамирович
Группа М3437 Факультет ИТиП

Направленность (профиль), специализация
Информатика и программирование

ВКР принята «____» _____ 20___ г.

Оригинальность ВКР _____%

ВКР выполнена с оценкой _____

Дата защиты «» г.

Секретарь ГЭК Штумпф С. А. _____

Листов хранения _____

Демонстрационных материалов/Чертежей хранения _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 6
1. Анализ предметной области 8
1.1. Система управления идентификацией Keycloak 8
1.1.1. Архитектура Keycloak 8
1.1.2. Регистрация SPI 8
1.1.3. Ключевые SPI и ProviderFactory для работы с базой данных 11
1.1.4. Механизм миграций и DBLockProvider. 11
1.1.5. Модель данных Keycloak 12
1.2. СУБД YDB 12
1.2.1. Ключевые свойства YDB 12
1.2.2. Диалект Hibernate для YDB. 14
1.3. Аналоги и существующие решения 14
Выводы по главе 1 15
2. Проектирование решения. 16
2.1. Общая архитектура модуля 16
2.2. Принципы проектирования схемы данных 17
2.2.1. Управление соединением с YDB. 17
2.2.2. Анализ подходов к миграции схемы 17
2.3. Разработка диалекта Hibernate для YDB 19
2.4. Архитектура механизма повторного выполнения транзакций. 19
Выводы по главе 2 19
3. Реализация 21
3.1. Диалект Hibernate v7 для YDB 21
3.2. Реализация провайдеров Keycloak. 21
3.2.1. YdbConnectionProviderFactoryImpl. 21
3.2.2. Обработка транзакций 22
3.2.3. Переопределение провайдеров сущностей. 22
3.2.4. Распределённые блокировки (YdbDBLockProvider). 22
3.3. Управление схемой данных через Liquibase 23
3.4. Переопределение именованных запросов. 23
Выводы по главе 3 24

4. Тестирование 25
4.1. Интеграционное тестирование 25
4.1.1. Тестирование распределённых блокировок 25
4.2. Нагрузочное тестирование 25
ЗАКЛЮЧЕНИЕ 33

ВВЕДЕНИЕ

В современных распределённых системах управление идентификацией и доступом (Identity and Access Management, IAM) является важным компонентом инфраструктуры. Keycloak — IAM-система с открытым исходным кодом, разработанная компанией Red Hat, — предоставляет единый вход (SSO), поддержку протоколов OAuth 2.0, OpenID Connect и SAML, управление пользователями и двухфакторную аутентификацию. Keycloak широко применяется в современных приложениях как готовое решение для обеспечения безопасности приложений.

Keycloak поддерживает только реляционные СУБД: PostgreSQL, MySQL, MariaDB, Oracle, TiDB и Microsoft SQL Server. По мере роста нагрузки на систему узким местом становится хранилище данных. Традиционные РСУБД масштабируются преимущественно вертикально и требуют значительных усилий для обеспечения отказоустойчивости в распределённых сценариях.

YDB (Yandex Database) — распределённая NewSQL-СУБД с открытым исходным кодом, разработанная Яндексом. В отличие от большинства распределённых СУБД, жертвующих согласованностью ради масштабируемости, YDB обеспечивает горизонтальное масштабирование и высокую доступность *одновременно* с полноценными ACID-транзакциями уровня изоляции *Serializable*. Это делает YDB привлекательной альтернативой для организаций, которым требуется легко масштабируемое хранилище без отказа от транзакционных гарантий.

Вместе с тем официальной поддержки YDB в Keycloak не существует: отсутствует совместимый Hibernate-диалект для актуальной версии фреймворка, а ряд конструкций, генерируемых Keycloak, несовместим с диалектом YQL.

Целью работы является разработка программного модуля, обеспечивающего интеграцию Keycloak с YDB в качестве основного хранилища данных.

Практическая значимость работы состоит в том, что разработанный модуль позволяет организациям, уже использующим YDB в своей инфраструктуре, развернуть Keycloak без необходимости поддерживать отдельную реляционную СУБД.

Новизна работы заключается в разработке открытого решения, обеспечивающего интеграцию Keycloak с YDB через механизм SPI.

Работа состоит из четырёх глав. В главе 1 проведён анализ предметной области: рассмотрена архитектура Keycloak и механизм SPI, описана СУБД YDB и её свойства, выполнен обзор существующих решений по интеграции Keycloak с нереляционными хранилищами. В главе 2 описано проектирование решения: общая архитектура модуля, подход к разработке Hibernate-диалекта и механизм повторного выполнения транзакций. В главе 3 представлена реализация: диалект Hibernate v7, SPI-провайдеры и механизм переопределения именованных запросов. В главе 4 описано тестирование: интеграционные тесты на тестовом контейнере YDB и нагрузочное тестирование с помощью keycloak-benchmark.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Система управления идентификацией Keycloak

С технической точки зрения Keycloak представляет собой сервер, которому приложения делегируют задачи аутентификации, авторизации и управления пользователями, благодаря чему приложения не хранят учётные данные пользователей и не реализуют собственную логику. Так, при аутентификации по протоколу OpenID Connect приложение перенаправляет пользователя на Keycloak, после успешного входа получает *access token* и использует его для авторизации последующих запросов без повторного обращения к серверу.

1.1.1. Архитектура Keycloak

Keycloak реализован на основе Quarkus — Java-фреймворка для создания облачно-ориентированных приложений. Для взаимодействия с базой данных используется ORM-фреймворк Hibernate. Управление схемой базы данных осуществляется через Liquibase.

Центральным элементом расширяемости Keycloak является механизм `Service Provider Interface (SPI)`. SPI позволяет заменять внутренние реализации компонентов пользовательскими, сохраняя совместимость с остальными частями системы.

1.1.2. Регистрация SPI

Для подключения собственной реализации через механизм SPI необходимо выполнить следующие шаги.

Шаг 1. Создать класс SPI, реализующий интерфейс `org.keycloak.provider.Spi`. Класс описывает точку расширения — его имя, тип провайдера и тип фабрики.

Например, SPI для подключения к базе данных через JPA выглядит следующим образом:

```
public class JpaConnectionSpi implements Spi {

    @Override
    public boolean isInternal() {
        return true;
    }

    @Override
```

```
    public String getName() {
        return "connectionsJpa";
    }

    @Override
    public Class<? extends Provider> getProviderClass() {
        return JpaConnectionProvider.class;
    }

    @Override
    public Class<? extends ProviderFactory>
       getProviderFactoryClass() {
        return JpaConnectionProviderFactory.class;
    }
}
```

Шаг 2. Зарегистрировать SPI и его реализацию через стандартный механизм Java `ServiceLoader`. В ресурсы приложения создаются файлы в директории `META-INF/services/`, содержащие полные имена классов реализаций:

`META-INF/services/org.keycloak.provider.Spi`

```
org.keycloak.connections.jpa.JpaConnectionSpi
```

Шаг 3. Реализовать интерфейс `ProviderFactory`. Фабрика управляет жизненным циклом провайдера и обязана реализовать следующие методы:

```
public interface ProviderFactory<T extends Provider> {
    T create(KeycloakSession session);
    void init(Config.Scope config);
    void postInit(KeycloakSessionFactory factory);
    void close();
    String getId();
    default int order() { return 0; }
}
```

а) `create` — создаёт экземпляр провайдера для каждой сессии;

б) `init` — вызывается один раз при старте сервера;

в) `postInit` — вызывается после инициализации всех фабрик;

г) `close` — вызывается при остановке сервера;

д) `getId` — возвращает уникальный идентификатор реализации;

е) `order` — приоритет среди фабрик (по умолчанию 0).

Шаг 4. Реализовать интерфейс `Provider`.

Интерфейс `Provider` — базовый интерфейс любого провайдера в Keycloak.

```
public interface Provider {
    void close();
}
```

Метод `close()` вызывается по завершении `KeycloakSession` — то есть по окончании каждого HTTP-запроса или транзакции. В нём необходимо освобождать ресурсы, удерживаемые провайдером: закрывать соединения с базой данных, сбрасывать кэши, завершать транзакции.

Таким образом, жизненный цикл провайдера привязан к сессии: фабрика создаёт новый экземпляр через `ProviderFactory.create(session)` в начале каждой сессии, а Keycloak вызывает `close()` при её завершении.

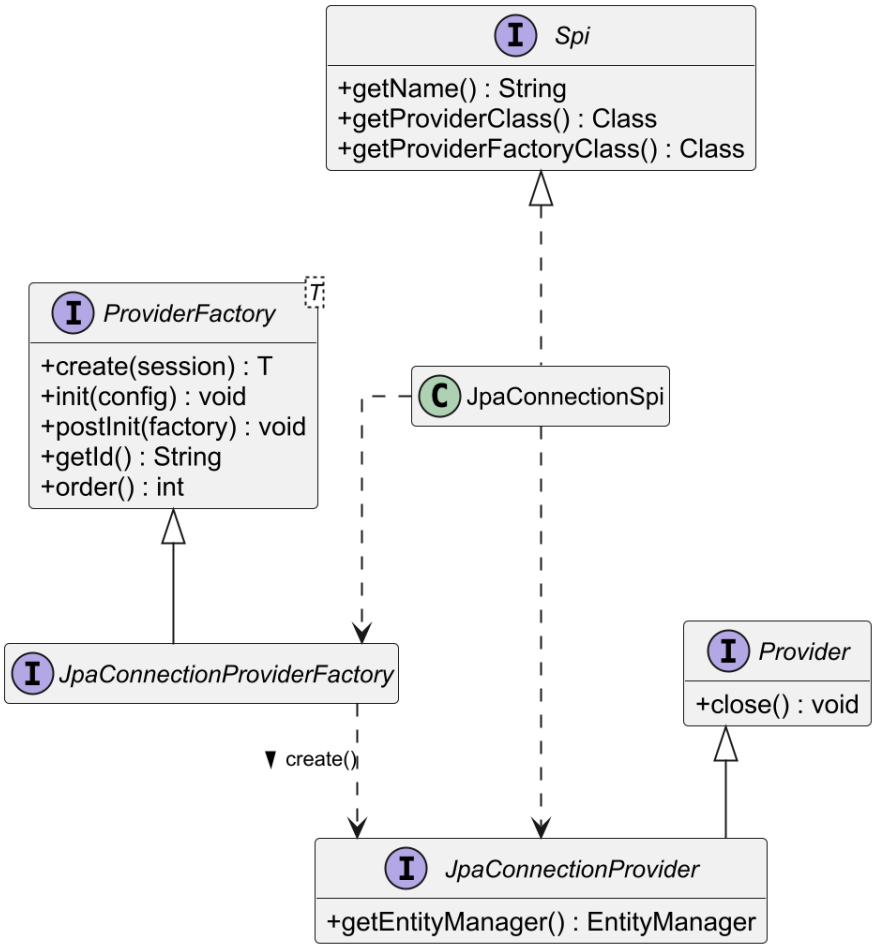

Рисунок 1 – Пример реализации SPI: JPA-подключение в Keycloak

Вся бизнес-логика реализуется не в наследниках интерфейса `Provider`. Например, `JpaRealmProvider` реализует сразу несколько интерфейсов —

`RealmProvider`, `ClientProvider`, `GroupProvider`, `RoleProvider` — и реализует всю работу с соответствующими сущностями на уровне базы данных через JPA. Каждый метод такого провайдера выполняет конкретную операцию: создание, поиск, обновление или удаление объектов в рамках текущей сессии и транзакции.

Выбор провайдера по приоритету

Алгоритм выбора следующий:

а) Если явно задан провайдер через конфигурацию, то используется он;
б) Если зарегистрирована только одна фабрика, то используется она;
в) Если зарегистрировано несколько фабрик, то выбирается та, у которой наибольшее значение `order()` среди всех с `order() > 0`.
г) Иначе используется фабрика с идентификатором default.

1.1.3. Ключевые SPI и ProviderFactory для работы с базой данных

— `JpaConnectionProviderFactory` — управляет созданием соединений с базой данных и жизненным циклом JPA `EntityManagerFactory`;
— `LiquibaseConnectionProviderFactory` — управляет подключением Liquibase для выполнения миграций;
— `DBLockProviderFactory` — реализует распределённую блокировку для синхронизации при параллельном старте нескольких инстанций Keycloak;
— `RealmProviderFactory`, `ClientProviderFactory`, `ClientScopeProviderFactory` и прочие — фабрики провайдеров для работы с основными сущностями данных.

1.1.4. Механизм миграций и DBLockProvider.

При старте Keycloak фабрика `JpaConnectionProviderFactory` инициирует создание схемы базы данных и выполнение миграций через Liquibase. Поскольку Keycloak может запускаться в виде нескольких параллельных узлов, необходима синхронизация: только один узел должен выполнять миграции в каждый момент времени. Эту роль выполняет `DBLockProvider` — провайдер глобальной распределённой блокировки, хранящий состояние в отдельной таблице базы данных.

Блокировки разделены по пространству имён (*namespace*):

— `DATABASE` — захватывается перед применением Liquibase-миграций (создание и обновление схемы таблиц);
— `KEYCLOAK_BOOT` — захватывается после завершения DDL-миграций для выполнения миграций данных (модельных миграций Keycloak).

Каждый узел ожидает снятия блокировки (по умолчанию до 900 секунд), после чего продолжает загрузку.

1.1.5. Модель данных Keycloak

Keycloak хранит данные в реляционных таблицах, отображаемых через Hibernate-аннотации на JPA-сущности.

Основные сущности Keycloak:

— **Realm** — область, содержащая конфигурацию, пользователей, клиентов, роли и группы;
— **Client** — OAuth/OIDC-клиент, зарегистрированный в realm;
— **ClientScope** — область видимости, определяющая набор claims в токене;
— **Role** — роль, назначаемая пользователям или клиентам;
— **Group** — группа пользователей с наследуемыми ролями;
— **UserEntity** — пользователь с атрибутами, учётными данными и привязкой к внешним провайдерам идентификации;

Схема Keycloak включает более 80 таблиц и активно использует внешние ключи для обеспечения ссылочной целостности. Это является принципиальным ограничением при переносе на YDB.

1.2. СУБД YDB

В данном разделе рассмотрены технические особенности YDB, непосредственно влияющие на проектирование интеграции с Keycloak.

1.2.1. Ключевые свойства YDB

Масштабируемость. YDB поддерживает горизонтальное масштабирование: при росте нагрузки система может автоматически перераспределять данные между узлами кластера без человеческого вмешательства.

Транзакционная модель. В отличие от большинства распределённых СУБД, жертвующих транзакционными гарантиями ради масштабируемости, YDB поддерживает полноценные ACID-транзакции на нескольких узлах кластера. YDB поддерживает сериализуемую изоляцию транзакций и использует *оптимистичную блокировку*: конфликты не предотвращаются превентивно,

а обнаруживаются в момент коммита. Если за время выполнения транзакции другая транзакция успела изменить затронутые строки, сервер откатывает её и возвращает ошибку `Transaction lock invalidated`.

YDB классифицирует ошибки на два типа. *Non-retryable*-ошибки указывают на логические нарушения (например, превышение схемных ограничений), и повтор бессмысленен. *Retryable*-ошибки носят временный характер: конфликт устранится сам собой, если приложение повторит транзакцию с нуля. Ошибка `Transaction lock invalidated` относится ко второму типу, однако повторяемая операция должна быть *идемпотентной* — повторный вызов не должен приводить к дублированию эффектов.

Отсутствие поддержки FOR UPDATE. YDB не реализует пессимистичные блокировки строк. Запросы с `SELECT ... FOR UPDATE` или `SELECT ... FOR SHARE` являются синтаксически некорректными в YQL.

Обязательный первичный ключ. Каждая таблица в YDB обязана иметь явно определённый первичный ключ. Таблицы без первичного ключа создать невозможно.

Отсутствие внешних ключей. YDB не поддерживает декларативные ограничения внешних ключей. Ссылочная целостность должна обеспечиваться на уровне приложения.

Диалект YQL. YDB использует YQL (Yandex Query Language) — диалект SQL с рядом ограничений:

— не поддерживаются ссылки на столбцы по порядковому номеру (`ORDER BY 1`);

— не поддерживается `CONCAT()`, вместо них используется оператор `||`;

— не поддерживаются операции `lower` и `upper`, вместо них используются `Unicode::ToLower` и `Unicode::ToUpper`

— каждый предикат в `JOIN ON` обязан ссылаться на столбцы *обеих* сторон соединения — условия фильтрации по одной таблице (вида `col = :param`) допускаются только в `WHERE`;

— неявные декартовы произведения (перечисление таблиц через запятую в `FROM`) не поддерживаются — вместо них требуется явный `CROSS JOIN`.

1.2.2. Диалект Hibernate для YDB

Hibernate — это популярный ORM-фреймворк на Java, который абстрагирует работу с базой данных от конкретного SQL-диалекта.Для взаимодействия с конкретной СУБД Hibernate использует *диалект* — класс, производный от `org.hibernate.dialect.Dialect`. Диалект отвечает за отображение Java-типов на SQL-типы конкретной СУБД и генерацию корректных SQL-выражений в её синтаксисе.

Для YDB существует реализация диалекта только для Hibernate 6, однако она содержит ряд неточностей в обработке YQL-специфичных конструкций. Поддержки Hibernate 7, используемого в Keycloak 26, не существовало.

1.3. Аналоги и существующие решения

На сегодняшний день существует реализация альтернативного хранилища для Keycloak на базе Apache Cassandra — проект `keycloak-cassandra-extension`.

Проект реализует замену стандартного JPA-хранилища на Apache Cassandra — распределённую NoSQL СУБД. Интеграция выполнена через механизм SPI Keycloak: расширение предоставляет собственные реализации `DatastoreProvider` и отдельных провайдеров для каждого типа сущностей (`UserProvider`, `RealmProvider`, `ClientProvider` и других). Для работы с базой данных используется напрямую DataStax Java Driver, без Hibernate и JPA.

Ключевым отличием является то, что Cassandra не поддерживает реляционные транзакции. Для обеспечения согласованности при конкурентных обновлениях используются *легковесные транзакции* (Lightweight Transactions, LWT). Каждая сущность хранит поле `version`, и обновление выполняется только при совпадении версии: если другой процесс успел изменить запись, операция отклоняется, и клиент должен повторить её.

Отсутствие реляционной модели потребовало полного переосмысления схемы данных. Если в стандартном Keycloak каждое поле сущности соответствует отдельной колонке таблицы (например, `JpaRealmEntity` содержит десятки типизированных колонок), то в `keycloak-cassandra-extension` большинство свойств сущностей хранится в обобщённых коллекциях — ассоциативных массивах атрибутов (`map<text, list<text»`). Это позволяет не менять схему при добав-

лении новых полей, но лишает возможности использовать SQL-поиск по конкретным полям. Для поддержки поиска пользователей создаются отдельные индексные таблицы по выбранным атрибутам; поиск по произвольным атрибутам требует полного обхода базы данных.

Проект поддерживает основные функции Keycloak, однако не реализует Authorization Services и организации (Organizations). Миграции схемы выполняются через библиотеку Cassandra Migration.

Таким образом, `keycloak-cassandra-extension` подтверждает принципиальную возможность замены реляционного хранилища в Keycloak через механизм SPI, однако потребовал полной переработки модели данных. Интеграция с YDB принципиально отличается: YDB является реляционной СУБД с JDBC-драйвером и поддержкой Hibernate, что позволяет сохранить существующую схему данных Keycloak. Основной задачей становится адаптация Hibernate-диалекта к YQL и обработка retryable-ошибок транзакций YDB.

Выводы по главе 1

В ходе анализа предметной области установлено, что Keycloak предоставляет механизм SPI, позволяющий заменить стандартные JPA и Liquibase-провайдеры пользовательскими реализациями. YDB поддерживает JDBC и Hibernate, что делает возможным сохранение существующей схемы данных Keycloak при интеграции. Существующие решения для нереляционных СУБД требовали полной переработки модели данных, тогда как подход с YDB позволяет ограничиться адаптацией диалекта и обработкой специфичных для YDB ошибок транзакций.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ РЕШЕНИЯ

2.1. Общая архитектура модуля

Модуль `keycloak-ydb-extension` встраивается в Keycloak через механизм SPI и состоит из трёх подмодулей Maven:

— `core/` — основная реализация расширения на Kotlin, упакованная как shaded JAR и размещаемая в директории `providers/` Keycloak;

— `retry-proxy/` — HTTP-прокси на Ktor, перехватывающий ответы с кодом 503 и повторяющий запрос при временных ошибках YDB;

— `test/` — интеграционные тесты с тестовым контейнером YDB.

Точкой входа служат файлы регистрации провайдеров в `META-INF/services/`, переопределяющие стандартные JPA- и Liquibase-провайдеры Keycloak.

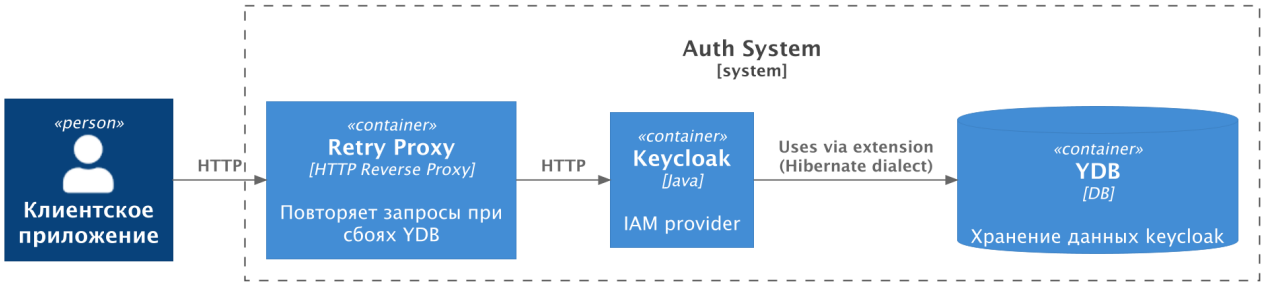

Рисунок 2 – Контейнерная диаграмма системы

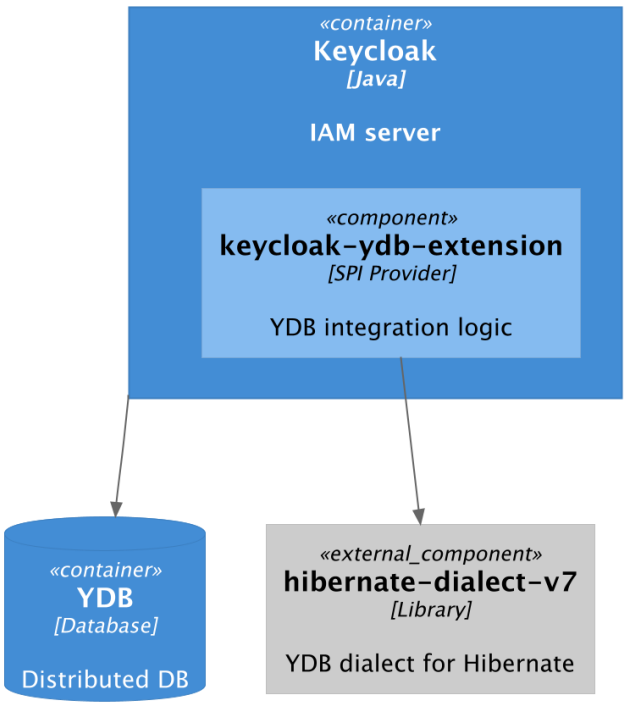

Рисунок 3 – Компонентная диаграмма модуля keycloak-ydb-extension

2.2. Принципы проектирования схемы данных

2.2.1. Управление соединением с YDB

Класс `YdbConnectionProviderFactoryImpl` является центральной точкой для обеспечения связи Keycloak с YDB. Он реализует интерфейс `JpaConnectionProviderFactory` и в отличие от стандартного провайдера Keycloak, который получает `EntityManagerFactory` через механизмы Quarkus с пулом соединений Agroal, данная реализация самостоятельно управляет пулом через HikariCP. Использование HikariCP обусловлено тем, что без явного пула соединений встроенный пул Hibernate быстро исчерпывается под нагрузкой и начинает возвращать ошибки. Agroal же недоступен вне контекста Quarkus. `EntityManagerFactory` создаётся с явно заданным диалектом и драйвером YDB. В методе `postInit()` выполняется применение миграций схемы с использованием распределённой блокировки `DBLockProvider`. Транзакции оборачиваются в `YdbJpaKeycloakTransaction`, который перехватывает retryable-ошибки YDB и преобразует их в HTTP 503.

2.2.2. Анализ подходов к миграции схемы

На начальном этапе были рассмотрены два подхода к созданию схемы базы данных.

Подход 1: стандартные Liquibase-миграции Keycloak. Keycloak хранит историю изменений схемы в виде наборов миграций Liquibase, накопленных за всю историю проекта (с версии 1.0). Набор включает более 70 XML-файлов, часть из которых содержит ветвление по типу СУБД (`dbms="postgresql"`, `dbms="mssql"`, `dbms="oracle"` и т. д.) и условные сценарии миграции с `onFail="MARK_RAN"`. YDB в этом списке не предусмотрен, поэтому при попытке применить стандартные миграции Liquibase выдаёт ошибки валидации. Миграции рассчитаны исключительно на поддерживаемые СУБД и активно используют конструкции, которые YDB не поддерживает:

— таблицы без явного первичного ключа (например, `SCOPE_MAPPING`, `REDIRECT_URIS`);

— добавление первичного ключа отдельной командой после `CREATE TABLE`;

— отсутствие `FOREIGN KEY`;

— ограничения `UNIQUE CONSTRAINT` — вместо них YDB поддерживает индексы `GLOBAL UNIQUE`;
— добавление `NOT NULL` на существующую колонку через `ALTER TABLE` — `NOT NULL` поддерживается только при создании таблицы;
— добавление значения по умолчанию на существующую колонку через отдельную операцию `addDefaultValue` — YDB поддерживает `DEFAULT` только при создании таблицы;
— операция `RENAME COLUMN`.

Подход 2: автоматическая генерация схемы через Hibernate (`HBM2DDL_AUTO=update`). Hibernate умеет генерировать DDL напрямую из аннотаций JPA-сущностей. Этот подход позволил создать некоторые таблицы, однако миграция не завершается успешно из-за того, что в таблицах не было первичиных ключей

Принятое решение: собственные Liquibase-миграции. Оба автоматических подхода оказались неприменимы без существенной доработки. Принято решение написать собственный набор миграций, воспроизводящих схему Keycloak с учётом ограничений YDB. За основу взята финальная схема PostgreSQL, полученная после применения всей истории стандартных Liquibase-миграций Keycloak.

При переносе схемы были применены следующие адаптации:

— **Составные первичные ключи.** Связующие таблицы, не имеющие суррогатного ключа в PostgreSQL-схеме (например, `SCOPE_MAPPING`, `REDIRECT_URIS`, `USER_ROLE_MAPPING`), получили составной первичный ключ из колонок, образующих естественный идентификатор строки.
— **Замена `UNIQUE`-ограничений на `GLOBAL UNIQUE`-индексы.** Все уникальные ограничения PostgreSQL заменены индексами вида `GLOBAL UNIQUE`, обеспечивающими ту же семантику уникальности в YDB.
— **Удаление `FOREIGN KEY`-ограничений.** Объявления внешних ключей удалены; ссылочная целостность обеспечивается на уровне приложения. Для сохранения производительности поисковых запросов на колонках, бывших внешними ключами, созданы обычные `GLOBAL`-индексы.

2.3. Разработка диалекта Hibernate для YDB

На начальном этапе предполагалось использовать существующий `hibernate-dialect-v6`, поскольку в зависимостях Keycloak явно упоминался Hibernate 6. Анализ зависимостей показал, что Keycloak 26 построен на Quarkus, который транзитивно подтягивает Hibernate 7. Исправления, внесённые в `hibernate-dialect-v6`, затрагивали API, несовместимый с Hibernate 7, и не применялись. Таким образом, для корректной работы с Keycloak 26 потребовалось создать отдельный модуль `hibernate-dialect-v7`.

Для обхода ограничения `FOR UPDATE` рассматривались два варианта: переопределение конкретных методов провайдеров и добавление в диалект настройки глобального игнорирования пессимистичных блокировок. Второй вариант принят как менее трудоёмкий и не требующий изменений в коде провайдеров Keycloak.

2.4. Архитектура механизма повторного выполнения транзакций

При конкурентном доступе YDB может вернуть retryable-ошибку (например `ABORTED` или `OVERLOADED`), означающую, что транзакцию необходимо повторить с нуля. Для её обработки разработана следующая архитектура: Keycloak при возникновении такой ошибки возвращает клиенту HTTP 503 с маркером `ydb_retryable` в теле ответа. Вместо прямых обращений к Keycloak клиенты направляют запросы к `retry-proxy` — HTTP-прокси, который проксирует их в Keycloak и при временном сбок YDB повторяет запрос автоматически, не возвращая ошибку клиенту.

Прокси отслеживает признак повторяемости по двум условиям одновременно: код ответа равен 503 и тело содержит строку `ydb_retryable`. Все остальные ответы передаются клиенту без изменений.

Между повторными попытками прокси выдерживает задержку по стратегии экспоненциального backoff, чтобы снизить нагрузку на сервер при пиковых конфликтах транзакций.

Выводы по главе 2

В ходе проектирования определена трёхкомпонентная архитектура модуля: `core` с SPI-провайдерами, `retry-proxy` для прозрачной обработки временных сбоев YDB и `test` с интеграционными тестами. Для поддержки

Hibernate 7 разработан отдельный диалект `hibernate-dialect-v7`. Механизм повторного выполнения транзакций реализован на уровне HTTP-прокси, что не требует изменений в клиентском коде и обеспечивает прозрачную обработку retryable-ошибок.

ГЛАВА 3. РЕАЛИЗАЦИЯ

3.1. Диалект Hibernate v7 для YDB

Модуль `hibernate-dialect-v7` реализует класс `YdbDialect`, расширяющий базовый диалект Hibernate 7 для работы с YDB. В ходе интеграции с Keycloak были выявлены и устранены следующие несовместимости.

Обработка OFFSET без LIMIT. Документация YDB описывает, что в случае когда в запросе присутствует `OFFSET`, то `LIMIT` не обязателен. На практике же запрос с `OFFSET` без `LIMIT` завершается ошибкой. Keycloak в ряде запросов указывает только `OFFSET`, что приводило к сбою при выполнении. В `YdbSqlAstTranslator` добавлена логика подстановки `LIMIT Long.MAX_VALUE` при наличии `OFFSET` без `LIMIT`.

Порядковые ссылки в ORDER BY. YDB не поддерживает конструкции вида `ORDER BY 1` (ссылка на столбец по порядковому номеру). В диалекте переопределён метод `supportsOrdinalSelectItemReference()`, что вынуждает Hibernate генерировать явные имена столбцов.

Строковые функции. YDB не поддерживает стандартные функции `lower()`, `upper()` и `concat()`. В диалекте они переопределены: `lower`/`upper` транслируются в `Unicode::ToLower`/`Unicode::ToUpper`, `concat` — в оператор `||`.

Игнорирование FOR UPDATE. Добавлена настройка `hibernate.dialect.ydb.ignore_lock_hints`. При значении `true` методы генерации SQL возвращают запрос без `FOR UPDATE`, что позволяет использовать Keycloak без переопределения провайдеров, вызывающих запросы с пессимистичными блокировками.

3.2. Реализация провайдеров Keycloak

3.2.1. YdbConnectionProviderFactoryImpl

Центральным компонентом расширения является `YdbConnectionProviderFactoryImpl`, реализующий интерфейс `JpaConnectionProviderFactory`. Именно через этот интерфейс Keycloak получает доступ к хранилищу данных. В методе

postInit() выполняется запуск Liquibase-миграций и создание EntityManagerFactory с YDB-диалектом. При создании каждого провайдера EntityManager оборачивается двумя прокси: стандартным прокси Keycloak и YdbEntityManagerProxy, который перехватывает retryable-ошибки YDB и возвращает HTTP 503 для повторного выполнения запроса. В качестве пула соединений используется HikariCP: встроенный пул Hibernate при исчерпании соединений немедленно выбрасывает исключение, тогда как HikariCP помещает запросы в очередь и ожидает освобождения соединения.

3.2.2. Обработка транзакций

Для корректной обработки ошибок YDB при фиксации транзакций реализован YdbJpaKeycloakTransaction, перехватывающий исключения при коммите и классифицирующий их как retryable с помощью утилиты YdbErrorUtils. При обнаружении retryable-ошибки клиенту возвращается HTTP 503, сигнализируя о необходимости повторного выполнения запроса.

3.2.3. Переопределение провайдеров сущностей

Поскольку YDB не поддерживает пессимистичные блокировки (FOR UPDATE), на начальном этапе был реализован YdbRealmProvider, наследующий JpaRealmProvider, с ручным каскадным удалением сущностей в методах removeRealm(), removeClient() и removeClientScope(). Однако в дальнейшем было принято решение игнорировать FOR UPDATE непосредственно в Hibernate-диалекте, что позволило отказаться от переопределения отдельных методов и упростить реализацию.

3.2.4. Распределённые блокировки (YdbDBLockProvider)

Стандартный DBLockProvider в Keycloak захватывает блокировку через SELECT FOR UPDATE, что недоступно в YDB. В рамках данной работы реализован YdbDBLockProvider, который захватывает блокировку путём прямой записи в таблицу DATABASECHANGELOGLOCK: при вызове waitForLock(namespace) в таблицу записывается информация о захваченном пространстве имён, после чего другие вызовы waitForLock с тем же пространством имён не могут получить блокировку.

Keycloak разделяет блокировки по пространствам имён: DATABASE для миграций схемы и KEYCLOAK_BOOT для инициализации модели данных.

Каждое пространство имён соответствует отдельной строке в таблице блокировок, что позволяет узлам кластера независимо управлять разными фазами запуска.

Для генерации корректного SQL под YDB реализованы кастомные компоненты на основе интерфейсов Liquibase: `YdbLockStatement` — описание операции захвата блокировки, и `YdbLockSqlGenerator` — генератор SQL-запроса, обновляющего строку таблицы блокировок для указанного пространства имён.

3.3. Управление схемой данных через Liquibase

Для управления схемой базы данных реализован `YdbLiquibaseConnectionProvider`, наследующий `DefaultLiquibaseConnectionProvider`. Он переопределяет путь к файлу миграций: вместо стандартного changelog Keycloak используется `ydb/db.changelog-master.xml`, а в качестве адаптера базы данных применяется `YdbDatabase` — кастомная реализация интерфейса Liquibase для YDB.

Схема базы данных адаптирована с учётом ограничений YDB: ограничения `UNIQUE CONSTRAINT` заменены на `GLOBAL UNIQUE INDEX`, внешние ключи (`FOREIGN KEY`) и ограничения `NOT NULL` через `ALTER TABLE` исключены, поскольку YDB не поддерживает их добавление после создания таблицы. Значения по умолчанию задаются только при создании таблицы в `CREATE TABLE`.

Дополнительно настроен порог создания индексов (`indexCreationThreshold`), по умолчанию равный 300 000 строк — параметр оптимизации для работы с YDB при больших объёмах данных.

3.4. Переопределение именованных запросов

Ряд стандартных JPQL-запросов Keycloak содержит синтаксические конструкции, несовместимые с YQL. Выявлено два класса проблем.

Первый — неявный `CROSS JOIN` через `SELECT FROM A, B`: Keycloak использует запросы, перечисляющие несколько сущностей через запятую в `FROM` и связывающие их условием в `WHERE`, что YDB не поддерживает. Такие запросы переписаны с явным `INNER JOIN`.

Второй — некорректные предикаты в `JOIN ON`: YDB требует, чтобы каждое условие в `ON` ссылалось на столбцы обеих соединяемых таблиц.

Keycloak размещает в `ON` фильтры вида `column = :param`, что вызывает ошибку в YQL. Такие условия перенесены в `WHERE`.

Механизм переопределения использует стандартный механизм Keycloak: файл `META-INF/queries-ydb.properties` содержит исправленные варианты именованных JPQL-запросов. При старте метод `addSpecificNamedQueries()` в `YdbConnectionProviderFactoryImpl` загружает эти запросы и регистрирует их в `EntityManagerFactory`, заменяя стандартные.

Выводы по главе 3

В ходе реализации разработан диалект Hibernate 7 с устранёнными несовместимостями YQL, реализованы SPI-провайдеры подключения к базе данных и управления схемой, а также механизм переопределения именованных JPQL-запросов для обхода конструкций, неподдерживаемых YDB. Совокупность реализованных компонентов обеспечивает работу Keycloak с YDB в качестве основного хранилища данных.

ГЛАВА 4. ТЕСТИРОВАНИЕ

4.1. Интеграционное тестирование

Для верификации корректности реализованных провайдеров разработан набор интеграционных тестов. Тесты запускаются против тестового контейнера YDB, что исключает расхождение между поведением в тестах и в реальной среде. Всего реализовано 47 тестовых случаев, охватывающих следующие области:

— операции с областями (`RealmModelTest`);
— управление клиентами и областями клиентов (`ClientModelTest`, `ClientScopeModelTest`, `ClientScopeStorageTest`);
— управление ролями и группами (`RoleModelTest`, `GroupModelTest`);
— одноразовые объекты и временны́е смещения (`SingleUseObjectModelTest`, `TimeOffsetTest`);
— транзакционное поведение при retryable-ошибках (`StorageTransactionTest`);
— события и аудит (`EventQueryTest`, `AdminEventQueryTest`).

4.1.1. Тестирование распределённых блокировок

Отдельное внимание уделено тестированию `YdbDBLockProvider`. В классе `DBLockTest` проверяется корректность работы блокировок при параллельном запуске нескольких потоков. Тест `testLockConcurrentlyGeneral` запускает 20 потоков, каждый из которых в 2 итерации захватывает и освобождает блокировку — семафор внутри теста гарантирует, что в каждый момент времени критическую секцию проходит ровно один поток. Тест `testTwoLocksCurrently` проверяет независимость блокировок по разным пространствам имён: потоки с `DATABASE` и `KEYCLOAK_BOOT` могут работать одновременно, не блокируя друг друга. Тест `testTwoNestedLocksCurrently` проверяет вложенные блокировки: часть потоков захватывает два уровня блокировки последовательно, остальные — только верхний уровень.

4.2. Нагрузочное тестирование

Для оценки производительности реализованного решения разработан стенд нагрузочного тестирования на основе инструмента

`keycloak-benchmark`, использующего Gatling в качестве движка генерации нагрузки.

Стенд поддерживает три конфигурации инфраструктуры:

— Keycloak + локальный YDB (Docker) с retry-proxy;
— Keycloak + удалённый YDB с retry-proxy;
— Keycloak + PostgreSQL — для сравнительного анализа.

Перед запуском тестов скрипт `setup-test-realm.py` создаёт тестовые клиенты, роли, группы и пользователи. Нагрузочный тест запускается командой:

```
./run.sh <scenario> <users-per-sec> [duration-sec]
```

Для нагрузочного тестирования выбраны сценарии, покрывающие два ключевых класса операций Keycloak. Первый класс — **аутентификация**: это наиболее часто выполняемые операции в production-системе, поскольку каждое обращение клиентского приложения к защищённому ресурсу инициирует хотя бы одно взаимодействие с Keycloak. Второй класс — **управление пользователями**: операции создания, поиска и удаления пользователей характеризуются интенсивной записью в основные таблицы YDB и вторичные индексы, что делает их показательными с точки зрения поведения базы данных под конкурентной нагрузкой.

В рамках данной работы протестированы следующие сценарии:

Таблица 1 – Сценарии нагрузочного тестирования

Сценарий	**Описание**
`CreateUsers`	Создание пользователя и получение списка пользователей
`CreateDeleteUsers`	Создание, получение списка и удаление пользователя
`ClientSecret`	Аутентификация по Client Credentials Flow
`AuthorizationCode`	Аутентификация по Authorization Code Flow
`LoginUserPassword`	Аутентификация через браузерную форму логина

Результаты каждого прогона сохраняются в директорию `results/` в виде HTML-отчётов Gatling с метриками пропускной способности (RPS), времени ответа (p50, p95, p99) и долей ошибок.

В рамках данной работы проведено предварительное нагрузочное тестирование на локальной машине разработчика с YDB, запущенной в Docker-контейнере. Тестирование на выделенном стенде с полноценным экземпляром YDB запланировано как следующий этап работы.

Для сценария `CreateUsers` выполнено четыре прогона. Сценарий включает три шага на итерацию: получение токена сервисного аккаунта, создание пользователя и получение списка пользователей. Прогоны 1 и 2 выполнены с синхронным индексом (`GLOBAL`), прогоны 3 и 4 — после оптимизации схемы (`GLOBAL ASYNC`).

Таблица 2 – Результаты сценария CreateUsers

Прогон	**Индекс**	**users/sec**	**Запросов**	**Ошибок**	**RPS**	**p50**
Прогон 1	GLOBAL	30	5 625	0%	87	11 мс
Прогон 2	GLOBAL	35	6 250	5,1%	51	2 818 мс
Прогон 3	GLOBAL ASYNC	35	6 561	0%	101	10 мс
Прогон 4	GLOBAL ASYNC	45	8 436	0%	128	15 мс

Первый прогон завершился успешно: все 5 625 запросов выполнены без ошибок, пропускная способность составила 87 запросов в секунду, медианное время ответа — 11 мс, 99-й перцентиль — 21 мс.

Второй прогон при увеличении нагрузки до 35 пользователей в секунду показал 5,1% ошибок и резкую деградацию латентности: медианное время ответа выросло до 2 818 мс, 99-й перцентиль — до 59 085 мс, часть запросов превысила 60-секундный таймаут Gatling. Анализ логов Keycloak выявил конкретную причину — конфликты блокировок на вторичном индексе таблицы пользователей:

```
Transaction locks invalidated.
Table: /local/USER_ENTITY/idx_user_service_account/indexImplT
```

При создании пользователей все параллельные транзакции одновременно обновляют один и тот же физический файл вторичного индекса `idx_user_service_account`, что приводит к инвалидации блокировок YDB (аналог `ABORTED`). Реализованный retry-proxy выполнял до 10 повторных попыток с экспоненциальной задержкой, однако под нагрузкой повторные попытки сами увеличивали конкуренцию на индексе, создавая положительную обратную связь. После исчерпания попыток proxy возвращал 502, которые Gatling фиксировал как ошибки; наиболее долгие запросы (до 60 с) завершились таймаутом на стороне Gatling в ожидании ответа proxy.

Для устранения проблемы индекс `idx_user_service_account` был переведён из синхронного режима (`GLOBAL`) в асинхронный (`GLOBAL`

ASYNC). В YDB синхронный индекс обновляется в рамках той же транзакции, что и основная таблица: при параллельных вставках все потоки конкурируют за одни и те же строки одного partition индекса по REALM_ID, провоцируя лавину конфликтов. Асинхронный индекс обновляется отдельно от основной транзакции — конкуренция на файле индекса исчезает. Для idx_user_service_account подобная задержка допустима: индекс используется исключительно для поиска сервисных аккаунтов, а не в критичном пути аутентификации.

Третий прогон с GLOBAL ASYNC при той же нагрузке 35 пользователей в секунду завершился без единой ошибки: 6 561 запрос, медианное время — 10 мс, 99-й перцентиль — 61 мс, максимальное время — 3 093 мс. Логи Keycloak фиксировали единичные retryable-ошибки на основной таблице USER_ENTITY, которые retry-механизм (YdbJpaKeycloakTransaction) разрешал прозрачно — без эскалации до ошибок на стороне клиента. Максимальное время ответа 3 093 мс соответствует одному полному циклу повторной попытки с экспоненциальной задержкой.

Четвёртый прогон подтвердил устойчивость оптимизации при дальнейшем увеличении нагрузки до 45 пользователей в секунду: 8 436 запросов, 0 ошибок, медианное время — 15 мс, 99-й перцентиль — 283 мс. Среднеквадратическое отклонение 143 мс при среднем 28 мс указывает на наличие единичных длительных повторных попыток (максимум 5 366 мс), однако все они завершились успешно в рамках retry-бюджета. Оба SLA — доля ошибок не более 0% и среднее время ответа не более 300 мс — выполнены.

Для сценария AuthorizationCode выполнено два прогона. Сценарий моделирует полный цикл аутентификации пользователя по протоколу Authorization Code Flow и включает четыре шага на итерацию: открытие страницы входа, ввод логина и пароля, обмен кода авторизации на токен и выход из системы.

Таблица 3 – Результаты сценария AuthorizationCode (YDB в Docker, локальная машина)

Прогон	**users/sec**	**Запросов**	**Ошибок**	**RPS**	**p50**	**p99**
Прогон 1	90	22 500	0%	341	19 мс	93 мс
Прогон 2	100	25 000	0%	338	1 686 мс	3 811 мс

Первый прогон при 90 пользователях в секунду завершился полностью успешно: все 22 500 запросов выполнены без ошибок, медианное время ответа составило 19 мс, 99-й перцентиль — 93 мс, максимальное время ответа — 355 мс.

Второй прогон при увеличении нагрузки до 100 пользователей в секунду не дал ошибок, однако медианное время ответа выросло до 1 686 мс, 99-й перцентиль — до 3 811 мс, а 67% запросов превысили 1 200 мс. Пропускная способность при этом осталась практически неизменной (338 RPS), что указывает на достижение точки насыщения: система успевает обработать все запросы, но накапливает очередь.

Сценарий `AuthorizationCode` выдерживает значительно более высокую нагрузку, чем `CreateUsers`, по двум причинам. Во-первых, сценарий является преимущественно читальным: чтение конфигурации области и клиента, поиск пользователя по имени — всё это операции чтения без конкурентных изменений одних и тех же строк. Во-вторых, единственные операции записи — создание сессии и токена — используют уникальные UUID-идентификаторы, что полностью исключает конфликты транзакций в YDB и, как следствие, ошибки типа `ABORTED`. В сценарии `CreateUsers` все потоки конкурентно пишут в одни и те же таблицы пользователей, что при 40 пользователях в секунду уже приводит к лавинообразному нарастанию retryable-ошибок YDB.

Для сценария `ClientSecret` выполнено три прогона. Сценарий моделирует Client Credentials Flow: один POST-запрос на эндпоинт токена с `client_id` и `client_secret` без участия пользователя. Это наиболее лёгкий сценарий — нет браузерного флоу, кук и редиректов.

Таблица 4 – Результаты сценария ClientSecret (YDB в Docker, локальная машина)

Прогон	**users/sec**	**Запросов**	**Ошибок**	**RPS**	**p50**	**p99**
Прогон 1	200	12 500	0%	192	7 мс	112 мс
Прогон 2	400	25 000	0%	385	7 мс	18 мс
Прогон 3	800	50 000	42,8%	758	—	—

Прогоны при 200 и 400 пользователях в секунду прошли без ошибок. Примечательно, что при 400 req/s показатели оказались даже лучше, чем при 200: медианное время — 7 мс, 99-й перцентиль — 18 мс, стандартное откло-

нение — 7 мс. Это объясняется прогревом JVM и стабилизацией пула соединений HikariCP.

При увеличении нагрузки до 800 req/s возникли ошибки, однако их причина — не Keycloak и не YDB, а ограничение локальной машины: при таком количестве запросов в секунду заканчиваются свободные сетевые порты операционной системы, и Gatling не может открыть новые соединения (`BindException: Can't assign requested address`). Таким образом, реальный предел производительности сервера выше 400 req/s, но точно измерить его на одной машине, где работают и клиент, и сервер, не представляется возможным.

Для сценария `CreateDeleteUsers` выполнено четыре прогона при нагрузке 30 и 40 пользователей в секунду с типами индекса `GLOBAL` и `GLOBAL ASYNC`. Сценарий моделирует полный жизненный цикл пользователя: получение токена сервисного аккаунта, создание пользователя, получение списка пользователей и удаление созданного пользователя — четыре запроса на одну итерацию. Все прогоны выполнялись на базе данных, уже содержавшей данные пользователей, накопленные в предшествующих прогонах `CreateUsers`.

Таблица 5 – Результаты сценария CreateDeleteUsers (YDB в Docker, локальная машина)

Индекс	**users/sec**	**Запросов**	**Ошибок**	**RPS**	**p50**	**p99**
GLOBAL ASYNC	30	7 500	0%	114	10 мс	70 мс
GLOBAL ASYNC	40	9 996	≈0%	151	13 мс	1 432 мс
GLOBAL	30	6 219	10,4%	51	деградация	
GLOBAL	40	7 290	17,8%	54	деградация	

С `GLOBAL ASYNC` оба прогона завершились практически без ошибок: p50 — 10–13 мс, p99 — 70 мс и 1 432 мс соответственно. С `GLOBAL` на той же базе данных наблюдалась катастрофическая деградация: ошибки 10% и 18%, RPS упал с ожидаемых 114/151 до 51/54, среднее время ответа — свыше 11 секунд, часть запросов исчерпывала 60-секундный таймаут Gatling.

Причина — конкуренция операций при смешанной нагрузке. Шаг «Получение списка пользователей» (`List Users`) при заполненной таблице удерживает широкие читающие блокировки (`S-locks`). Параллельные `CREATE` и `DELETE` запрашивают эксклюзивные блокировки (`X-locks`) —

возникает конфликт, YDB отвечает ошибкой `ABORTED`. При `GLOBAL` каждая операция `CREATE` синхронно записывает в партицию индекса, что создаёт дополнительное конкурентное давление, которое retry-proxy уже не успевает компенсировать, и отказы распространяются на все типы запросов. `GLOBAL ASYNC` убирает синхронную запись в индекс из критического пути, снижая суммарное число конкурирующих транзакций — и retry-механизм справляется с оставшимися конфликтами прозрачно.

Для сценария `LoginUserPassword` выполнено два прогона. Сценарий моделирует аутентификацию через браузерную форму: открытие страницы логина и отправка формы с именем пользователя и паролем — два HTTP-запроса на итерацию. В отличие от `AuthorizationCode`, сценарий не выполняет обмен кода на токен и не завершает сессию logout'ом.

Таблица 6 – Результаты сценария LoginUserPassword (YDB в Docker, локальная машина)

Прогон	**users/sec**	**Запросов**	**Ошибок**	**RPS**	**p50**	**p99**
Прогон 1	90	11 250	0%	170	29 мс	203 мс
Прогон 2	200	22 744	19,1%	274	—	—

При 90 пользователях в секунду прогон завершился без ошибок: 11 250 запросов, среднее время ответа — 31 мс, 99-й перцентиль — 203 мс, все запросы уложились в 800 мс.

При увеличении нагрузки до 200 пользователей в секунду возникли ошибки (19,1%). Все ошибочные запросы вернули статус 502 и имели время ответа ровно 10 секунд (стандартное отклонение — 9 мс) — это в точности значение `connectTimeout` прокси-сервера. При перегрузке Keycloak перестаёт принимать новые TCP-соединения; прокси не может подключиться в течение 10 секунд и возвращает `502 Bad Gateway` без логирования (исключение перехватывается без retry и без записи в лог).

Важно понимать природу этого ограничения. В Keycloak онлайн-сессии не хранятся в базе данных — они живут в Infinispan кэше. Keycloak предоставляет две реализации `UserSessionProviderFactory`: `InfinispanUserSessionProviderFactory` (embedded-режим, Infinispan встроен в процесс Keycloak) и `RemoteUserSessionProviderFactory` (внешний Infinispan-кластер). Обе реализации основаны на Infinispan; замена этого слоя на произвольную

реляционную базу данных через SPI не предусмотрена. В базу данных (через JPA и, следовательно, в YDB) пишутся только **offline-сессии** через `UserSessionPersisterProvider` — эта часть в реализации присутствует (таблицы `OFFLINE_USER_SESSION` и `OFFLINE_CLIENT_SESSION`).

Таким образом, деградация при 200 req/s — это ограничение JVM-кучи и Infinispan, а не YDB. Сценарий `LoginUserPassword` без logout'а накапливает сессии в памяти процесса: 200 req/s $\times$ 35 с $\approx$ 7 000 объектов сессий в heap, что приводит к давлению на GC и насыщению пула потоков Keycloak. В реальном развёртывании сессии имеют TTL и периодически вытесняются Infinispan'ом, поэтому подобная деградация не возникала бы при стабильной нагрузке.

Проведённое нагрузочное тестирование в условиях локальной разработки показало, что ключевые сценарии Keycloak — аутентификация и управление пользователями — функционируют корректно и демонстрируют приемлемую производительность. Сценарии, основанные преимущественно на чтении, устойчиво выдерживают сотни запросов в секунду; сценарии с интенсивной записью ограничены конкуренцией транзакций YDB, которая эффективно снижается переходом на асинхронные вторичные индексы. Вместе с тем полученные результаты не отражают поведение системы в реальном production-окружении, ведь тестирование проводилось на одной машине с YDB в Docker-контейнере, что существенно ограничивает как ресурсы, так и масштаб нагрузки. Тестирование на выделенном стенде с полноценным кластером YDB, соответствующим production-конфигурации, запланировано как следующий шаг и позволит получить окончательные выводы о производительности решения.

ЗАКЛЮЧЕНИЕ

В рамках данной работы разработан программный модуль, обеспечивающий интеграцию Keycloak с YDB в качестве основного хранилища данных.

Проведён анализ архитектуры Keycloak и особенностей YDB, определяющих требования к адаптации: несовместимость ряда SQL-конструкций с диалектом YQL и необходимость обработки временных сбоев при конкурентных транзакциях. Реализован Hibernate-диалект, обеспечивающий корректную работу Keycloak с YDB, и набор SPI-провайдеров, отвечающих за управление соединениями, выполнение миграций схемы и распределённую блокировку при запуске. Разработан retry-механизм на уровне JDBC, прозрачно перехватывающий временные сбои транзакций YDB и повторяющий их без изменений в коде приложения. Выполнено нагрузочное тестирование на ключевых сценариях Keycloak, подтверждающее работоспособность решения и позволившее выявить направления дальнейшей оптимизации.

Разработанный модуль позволяет организациям, использующим YDB в своей инфраструктуре, развернуть Keycloak без необходимости поддерживать отдельную реляционную СУБД. Модуль опубликован в открытом доступе и является частью экосистемы YDB Java Dialects.

В качестве направлений дальнейшей работы планируется проведение нагрузочного тестирования на выделенном production-стенде с полноценным кластером YDB для получения объективных данных о производительности в реальных условиях эксплуатации.